

Multi-resolution semantic abstraction over an evolving knowledge hypergraph

Ilia Dzneldze · Intern assessment, Constructor Knowledge Labs · September 2026

I build a three-level hierarchy of concept clusters (12, 50 and 200 groups) for each of four yearly snapshots of the TKH export and track it through time, using average linkage on one graph that blends an arity-weighted clique expansion of the hypergraph with a semantic nearest-neighbour graph. Most of my effort went into evaluating it without grading it by its own ruler. Levels 1 and 2 are coherent to a blind reader, a blind rater found none of 48 glosses wrong, and routing through the labels beats chance. Level 0 isn't shown to be coherent, change between snapshots isn't localised, and the hypergraph carries about 7% of the affinity, not the 30% its weight suggests.

1 The data and its snapshots (T1)

The export has 5,798 nodes of 12 types and 1,429 hyperedges extracted from 52 papers on machine-learned interatomic potentials. I cut it by hyperedge year: the snapshot at t keeps every edge dated t or earlier and the nodes those edges reference. Articles and authors stay in as edge members but are never clustered, since neither is a concept.

Table 1. The four snapshots. *New* counts nodes added since the previous snapshot; *seen after t* counts concept nodes that an early edge references before the corpus first mentions them.

snapshot	edges	nodes	concepts	papers	new nodes	arity > 2	seen after t
≤ 2020	405	1,527	1,406	16		73%	22
≤ 2022	589	2,200	2,046	23	673	76%	35
≤ 2024	1,063	4,184	3,895	39	1,984	80%	19
≤ 2026	1,429	5,798	5,428	52	1,614	80%	0

The structure really is higher-order: median arity is 3 throughout and the largest edge grows from 45 to 65 concepts. The corpus nearly doubles between 2022 and 2024, which matters for stability later. Three data problems shape the rest. `origin_year` is missing for 86% of nodes, so nothing can depend on it. There is no entity resolution, so 307 surface forms repeat among concept nodes (“MAE” is a separate node in every paper that uses it). And because membership follows the edge year, an early edge can pull in a node the corpus hasn't seen yet. I first claimed the labeller's input was temporally honest by construction, and it isn't (§5.3).

2 Formal statement (Deliverable 0)

Construction. For a snapshot, let V be its concept nodes, $e_c \subseteq V$ the concept members of hyperedge e , $n_e = |e_c|$, and x_i the unit-norm MPNet embedding of node i 's surface form. For $i \neq j$,

$$S_{ij} = \sum_{e: i, j \in e_c} \frac{1}{n_e - 1}, \quad M_{ij} = \cos(x_i, x_j) \cdot \mathbf{1}[j \in \text{kNN}_{15}(i) \vee i \in \text{kNN}_{15}(j)], \quad (1)$$

$$A_{ij} = \alpha \tilde{S}_{ij} + (1 - \alpha) \tilde{M}_{ij}, \quad \alpha = 0.3, \quad d(C, D) = 1 - \frac{1}{|C||D|} \sum_{i \in C, j \in D} A_{ij}, \quad (2)$$

where $\tilde{S} = \min(S/q_{0.99}(S), 1)$, likewise $\tilde{M}$, and an entry missing from the sparse graph counts as $A_{ij} = 0$. Average linkage repeatedly merges the pair of groups with the smallest d , and level k is the cut of the resulting dendrogram with N_k groups, $(N_0, N_1, N_2) = (12, 50, 200)$. S is a weighted clique expansion: every member spreads one unit of weight over its co-members, so an edge of arity n adds $n/2$ in total rather than $\binom{n}{2}$, and one 65-concept table can't swamp the graph. With unit weights, edges above arity 10 would hold 94% of the pairwise weight; with $1/(n-1)$ they hold 63%.

A criterion, not an objective. Average linkage is greedy and optimises nothing globally, so what the method satisfies is a criterion: at every step, merge the two groups with the highest mean affinity, and read level k off that process at N_k groups. What the criterion buys is nesting. With missing entries read as 0, merge heights never decrease, so every cut refines the one above it and the levels are laminar by construction. Filling missing entries with anything more optimistic can cause inversions and break that silently. I checked the implementation instead of trusting it: on random sparse graphs it matches `scipy`'s dense average linkage in merge heights and cuts, and the same test fails for the weighted variant (WPGMA), so it isn't passing trivially.

The trade-off, and the alternative I rejected. One joint graph turns the balance between structure and meaning into a single number I can sweep, and lets a strong semantic tie override weak structure at any level. I rejected a two-stage scheme,

where structure fixes the coarse level and semantics refines inside it: two groups on the same topic that happen to share no hyperedge could then never meet at the top, and in a corpus of 52 papers that’s the normal case. I argued this rather than built it, which is a gap. What I got wrong is what α measures. The normalised graphs sit on different scales (median entry 0.06 for $\tilde{S}$, 0.57 for $\tilde{M}$), so at $\alpha = 0.3$ structure carries **6.6%** of the affinity mass, not 30%. It matters a little more than that, since 94% of structural pairs are ones the k -NN graph never proposes, but the honest description of what ships is semantic nearest-neighbour clustering with the hypergraph as a tie-breaker.

Across snapshots. Snapshots only add nodes ($V_t \subseteq V_{t+1}$) and each is clustered from scratch. A level- k cluster C at t links to $\sigma(C) = \arg \max_D J(C, D)$ at $t+1$ if $J \geq \tau = 0.15$, where $J(C, D) = |C \cap D| / |C \cup (D \cap V_t)|$, so new nodes don’t count against a match. A D linked from one C inherits its persistent id, from several it is a merge and keeps the largest one’s id, and from none it is a birth. A C with no link dies. A one-to-one link is a split if C also reaches τ with another cluster, and otherwise stable, grow or shrink by J and size change. Matching keeps each snapshot reproducible on its own, but nothing pushes the clustering toward stability, so P5 is measured, not enforced.

Complexity. Building S costs $\sum_e n_e^2$ (about 66k pairs at 2026), M is a 15-neighbour search, and the heap-based merge is roughly $O(E \log E)$ in the stored affinities while neighbour lists stay sparse. All four snapshots build in a few minutes on a laptop CPU, mostly spent embedding.

Table 2. Which of the brief’s properties hold by construction and which only empirically.

property	status	evidence
P1 laminar refinement	by construction	monotone merge heights; exact check at every snapshot (<code>validate_hierarchy.py</code>)
P2 size budget	by construction	at most N_k groups per cut; $N_0 = 12$ everywhere
P3 semantic coherence	empirical	blind intruder test passes at levels 1 and 2, not at level 0 (§5.1)
P4 hyperedge fidelity	partial	S is a projection; the T4 collapse runs and is written out, but doesn’t build the levels (§4)
P5 temporal stability	empirical, not enforced	cross-snapshot ARI 0.35 / 0.45 / 0.65 by level; change isn’t localised (§5.2)
P6 faithful labels	empirical, partial	0 of 48 glosses rated wrong blind; levels 0 and 1 labelled, level 2 not; input temporally honest for about 98% of nodes (§5.3)

3 Where the method sits

The weighting comes from work showing that hypergraph Laplacians reduce to clique and star expansions (Agarwal et al., 2006; Zhou et al., 2006). I use the arity-aware weights without the spectral step, so what I claim is a well-weighted projection, not a hypergraph-native method. For the hierarchy I chose plain agglomeration over principled hierarchical models. Clauset et al. (2008) fit a dendrogram as a generative model and score it on held-out links, the same logic as my held-out hyperedge test. Peixoto’s nested block model (Peixoto, 2014) picks the number of groups per level from the data. That would remove the fixed cut sizes I suspect cause global churn, but it gives up the hard budget P2. I rejected hypergraph modularity (Kumar et al., 2020) because it doesn’t give a laminar family without repair. Ruggeri et al. (2023) show hypergraph communities are hard to detect when pairs rarely recur across edges; here only 1.7% of concept pairs recur across papers, which I think is why structure generalises weakly. Loukas (2019) coarsens pairwise graphs, so it never has to decide what an edge spanning three groups becomes, which is the question T4 answers. SHyPar (Sajadinia et al., 2024) argues that local coarsening heuristics like mine lose global structure, a plausible but untested reason my multilevel variant collapsed. For time, the event vocabulary is from Greene et al. (2010), and my scheme is what Asgari et al. (2023) call No-Smoothing: cluster each snapshot, then match. They found it the least smooth option in most settings; my warm-start test found that the smoother option costs coherence on real data.

4 Time and hyperedge collapse (T3, T4)

T3. Snapshots are matched by the rule in §2, and `temporal_events.json` holds 1,091 events: 262 initial births at 2020, then 829 over the three transitions (268 grows, 152 births, 128 splits, 108 stable, 103 merges, 43 deaths, 27 shrinks). 85 of those births are split pieces, logged from both sides. Of the three thresholds I set by feel, only the match threshold is sensitive: moving it from 0.10 to 0.20 takes deaths from 12 to 78 and splits from 192 to 84, so every event keeps its raw Jaccard for filtering. A static picture of levels 0–2 at all four snapshots, with the level-0 flows between them, is in `outputs/figures/hierarchy_across_snapshots.png`.

T4. If all of a hyperedge’s endpoints land in one super-node, the edge becomes internal and is kept as a count. If they land in two, it becomes a pairwise edge. If they land in three or more, it stays one k -ary edge; exploding it into $\binom{k}{2}$ pairs would inflate density and turn one asserted fact into many. Coarse edges over the same super-nodes merge into one weighted edge with a relation-type breakdown. What the rule loses is which original member sat in which original edge. At level 0 in 2026 the 1,429 edges collapse to 128 pairwise and 464 k -ary coarse edges (up to arity 41), plus 7 internal ones, few because

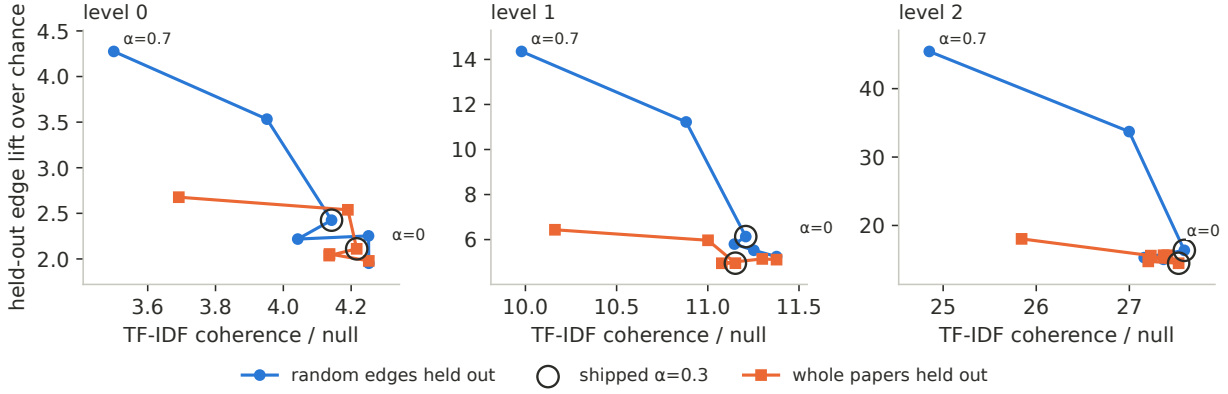


Figure 1. The structure/meaning trade-off across α on the 2026 snapshot: held-out hyperedge lift against TF-IDF coherence over its null, holding out random edges (blue) or whole papers (orange), 5 seeds. The ring marks the shipped $\alpha = 0.3$.

articles pass through as singletons. Exploding the k -ary ones would give 6,926 pairwise edges, about 15 per assertion. The rule runs at every level and is written to `hierarchy.json`, but the shipped levels are cuts of one dendrogram, so nothing reads it back. That falls short of the brief. The version that does use it clusters each coarser level on the collapsed hypergraph below. It lost a third or more of level-0 coherence and put about 40% of nodes in one cluster, and a size-normalised second attempt failed the same way, so under the rule I had set it didn't ship.

5 Evaluation

5.1 Coherence and the circularity hazard

Measuring coherence with the MPNet embeddings that built the clusters would grade the method with its own ruler, so I measured it three other ways, from weakest to strongest. The first is TF-IDF over the same text, against a random-labels null. At 2026 the clusters score 4.3, 11 and 28 times the null mean by level, and none of 30 random clusterings of the same sizes reaches them, but this is the weakest check, because TF-IDF reads the same strings: 69% of MPNet neighbour pairs share a TF-IDF term, against 7% of random pairs. A degree- and arity-preserving shuffle of the hypergraph gets almost the same TF-IDF coherence ($z = 0.3, 0.4$ and 2.9), which is what you'd expect when structure carries 6.6% of the mass.

The second check doesn't read text at all. I hide 20% of the hyperedges, cluster on the rest, and ask how often the members of a hidden edge share a cluster, relative to a random partition of the same sizes. At level 0, moving α from 0 to 0.3 to 0.5 raises that lift from 1.95 to 2.42 to 3.53. Holding out whole papers instead cuts it to 1.98, 2.11 and 2.54, so the structure term mostly encodes co-occurrence inside papers it has already seen. It clears the bar I set only at level 0 (paired gain $[0.01, 0.25]$), and no α beats 0.3 on both axes (Figure 1).

The third check is a blind intruder test in the style of Chang et al. (2009). A fresh LLM rater with no repo access sees five members of a super-node plus one type-matched node from another level-0 branch, and picks the odd one out. The rater (Claude Opus 5.5) and the labeller (Claude Sonnet 5) are one model family and may share blind spots. Chance is 1 in 6. On null items built from random nodes it found the intruder once in 20, so it wasn't reading type or length. On real items it found it 17 of 23 times at level 2 and 14 of 24 at level 1, both well above chance, but only 4 of 12 times at level 0 ([14%, 61%]), which fails the rule I set (Figure 2, left). My conclusion is that levels 1 and 2 hold together for a blind reader and level 0, which is also the least stable level, is not shown to. With only 12 level-0 groups there is no bigger sample to be had.

5.2 Stability

Rebuilding five times with 10% of hyperedges removed gives ARI 0.58, 0.77 and 0.91 by level. I don't read that as robustness: the perturbation only touches the structural graph, so it rewards ignoring the hypergraph (0.75, 0.92 and 0.98 at $\alpha = 0.05$), and I partly chose α on it. Cross-snapshot ARI on shared nodes is 0.35, 0.45 and 0.65, with intervals from resampling half the shared nodes (Figure 3a). My first interval resampled with replacement and came out entirely above its own estimate at level 2, because duplicated nodes add agreeing pairs. Per transition, the level-1 intervals don't overlap: 0.52, then 0.37 over 2022 to 2024 as the corpus nearly doubled, then 0.47. The transitions really differ, and the mean over three is only a summary.

P5 also asks that change be localised to where the corpus changed. It isn't, at least not where new hyperedges land: at no level does churn rise with a cluster's share of new edges (Spearman $-0.32, +0.03, -0.01$, intervals spanning 0), and 351 fine clusters with under 2% new edges still churn 0.44. Churn does follow the share of new *semantic* neighbours at levels 1 and 2 (Spearman 0.42 and 0.18), but I found that only after the planned test failed, so it's a lead. My guess is that fixed cut sizes

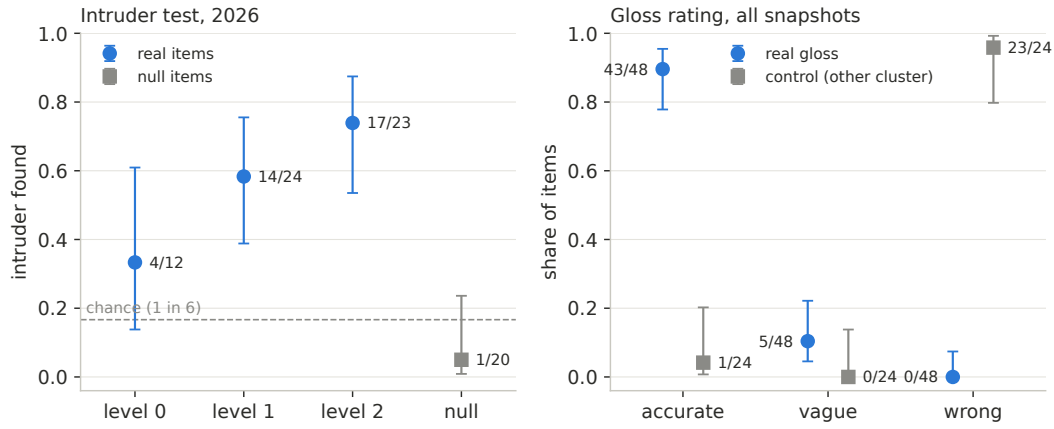
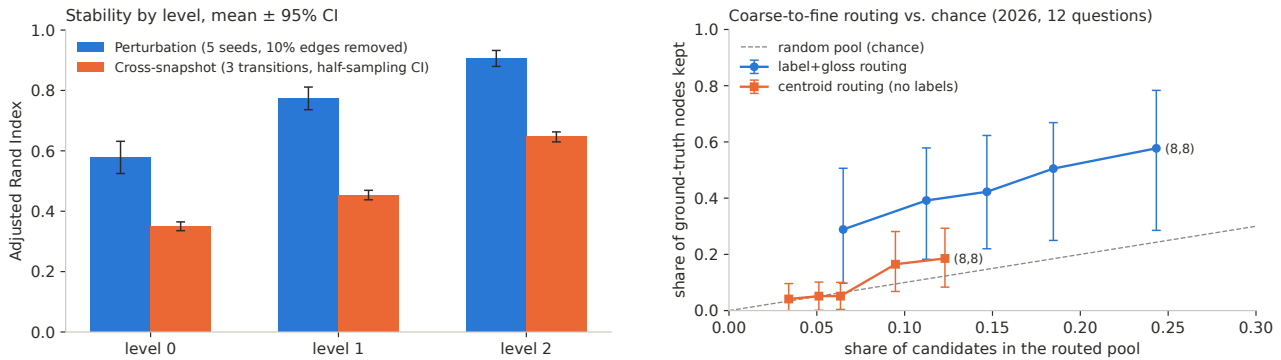


Figure 2. Blind judgements with Wilson 95% intervals. Left: intruder detection by level, against chance and against null items. Right: ratings of glosses shown with their own cluster’s unseen members (blue) or another cluster’s (gray).



(a) Perturbation (5 seeds) and cross-snapshot stability (half-sampling interval over shared nodes).

(b) Share of ground-truth nodes kept against the routed pool’s share of candidates, with bootstrap intervals over 12 questions.

Figure 3. Stability by level (a) and coarse-to-fine routing against a random pool of the same size (b).

force distant clusters to merge whenever a new region needs room. A warm-started variant raised level-2 cross-snapshot ARI from 0.65 to about 0.9 at a real but noisy coherence cost (level-2 $\approx$ 7% to 18% lower on average), so it didn’t ship, though it came closer than I first reported.

5.3 Label faithfulness

The 248 labels and glosses cover levels 0 and 1; level 2 has none, which is short of P6’s “every super-node”. They were written by fresh LLM agents from a seeded sample of 25 members plus the type counts of all members, and the exact prompts are in `labeling_input.json`. An earlier set, written by agents that could see the question files, was replaced. The NLI check grades each gloss against up to 15 members the labeller never saw, with another cluster’s members as the control. Contradiction runs 5% to 14% by snapshot (controls 45% to 63%), and “not entailed” runs 60% to 79%. I didn’t know which of those was the over-claim rate, so I had a blind rater mark 72 items as accurate, vague or wrong, real and control mixed (Figure 2, right). It rated **0 of 48 real glosses wrong** ([0%, 7%]) and 23 of 24 controls wrong, and it called 32 of the 36 real glosses NLI marks “not entailed” accurate. Under the rule I had set, the not-entailed rate measures a weak premise, not over-claiming. NLI contradiction agrees with the rater only moderately ($\kappa = 0.46$, [0.24, 0.68]) and raises false alarms. Grading glosses against their members’ source-paper titles, which neither the labeller nor the clustering saw, points the right way (21% contradicted against 31% for controls), but the intervals overlap, so I don’t lean on it. Temporal honesty holds for the labeller’s input for about 98% of nodes, not by construction: the 2020 label `L1_S00048` names EquiformerV2, which the corpus first saw in 2024.

5.4 Extrinsic: coarse-to-fine retrieval

For each question, the drill-down routes through label and gloss text (the top 8 of 12 level-0 groups, then the top 8 of their level-1 children) and ranks the nodes in that pool; the flat baseline ranks all 3,104 method-like nodes by cosine to the question. Of the 18 questions (the brief says 17), I score the 14 of type A, which list expected methods. The 4 of type B and every `method_claims` entry stay unmapped, so supporting claims are unscored. Ground-truth method names resolve to nodes by exact or substring match. My first matcher let element nodes (“N”, “S”, “Si”) match names like “physics-informed”,

so a third of the answer key was chemical elements, and the numbers looked plausible the whole time. Requiring both sides of a substring match to be at least four characters leaves 12 of the 14 with at least one matched node. Routing works: the pool keeps 58% of ground-truth nodes while covering 24% of candidates, a lift of 0.33 over a same-size random pool ([0.04, 0.54]), and every budget clears chance. Routing on member centroids instead of labels doesn't (Figure 3b), so the signal is in the labels. Ranking doesn't improve: recall@20 is 0.121 against flat's 0.119 in-sample, and with settings chosen by leave-one-out the paired difference is -8.7 points $([-25.9, +0.7], p = 0.37)$, resting on 4 against 5 retrieved ground-truth nodes. The hierarchy finds the right region, and the ranking inside it is the weak part.

6 Verified and assumed

Verified, each with a check or a number behind it	Assumed or unresolved
Laminarity at every snapshot, by an exact validator	The two-stage alternative: argued, not built
The UPGMA implementation, against scipy	Blind judgements from one LLM rater, of the labeller's model family; no second rater, no human pass
Coherence against two nulls, held-out hyperedges and a blind intruder test	Why α near 0.3 works, as opposed to that it does
Stability intervals by half-sampling, with the old estimator's bias pinned in a test	The temporal thresholds, set by feel; the match threshold is sensitive
The NLI judge, against blind gloss ratings	Whether fixed cut sizes cause the global churn
The structural share of the affinity (6.6%), measured	The 7 substring matches in the answer key, unchecked against the papers
The ground-truth matcher, fixed and pinned by a test	Level-2 labels, which don't exist
A one-command rerun from a bare Python that reproduces all 45 outputs byte for byte	The Linux install, not run end to end

Reproducing it. `py -3.11 scripts/reproduce_all.py` goes from a fresh clone to every result and figure, building its own environment first. From a folder with no environment at all it reproduced all 45 output files byte for byte against the published checksums in 72 minutes on a laptop CPU. The same text can embed differently in the last bits on other hardware, so I publish every model output from my run, and a different result elsewhere can be traced to the embeddings or to the code. A reproducer can also swap in their own labels and blind ratings and rerun the whole evaluation on them.

7 With four more weeks

On the method, I'd normalise the two graphs so that α means influence and re-sweep it, and cut at a fixed merge height instead of fixed counts, which I suspect drives the global churn. On the evaluation, a human rater for the intruder and gloss tasks, a semantic-side perturbation so that stability stresses both signals, and the two-stage alternative measured rather than argued. On the extrinsic task, mapping the type-B questions and method claims to nodes so that there are more than 12 questions, and then a better ranker inside the routed pool, since that's where the hierarchy currently loses what routing gains.

References

- S. Agarwal, K. Branson, S. Belongie. Higher order learning with graphs. *ICML*, 2006.
- Y. Asgari, R. Cazabet, P. Borgnat. Mosaic benchmark networks: modular link streams for testing dynamic community detection algorithms. arXiv:2310.02840, 2023.
- J. Chang, J. Boyd-Graber, C. Wang, S. Gerrish, D. Blei. Reading tea leaves: how humans interpret topic models. *NIPS*, 2009.
- A. Clauset, C. Moore, M. E. J. Newman. Hierarchical structure and the prediction of missing links in networks. *Nature* 453, 98–101, 2008.
- D. Greene, D. Doyle, P. Cunningham. Tracking the evolution of communities in dynamic social networks. *ASONAM*, 176–183, 2010.
- T. Kumar, S. Vaidyanathan, H. Ananthapadmanabhan, S. Parthasarathy, B. Ravindran. Hypergraph clustering by iteratively reweighted modularity maximization. *Applied Network Science* 5:52, 2020.
- A. Loukas. Graph reduction with spectral and cut guarantees. *JMLR* 20(116), 2019.
- T. P. Peixoto. Hierarchical block structures and high-resolution model selection in large networks. *Physical Review X* 4, 011047, 2014.
- N. Ruggeri, A. Lonardi, C. De Bacco. Message-passing on hypergraphs: detectability, phase transitions and higher-order information. arXiv:2312.00708, 2023.
- H. Sajadinia, A. Aghdaei, Z. Feng. SHyPar: a spectral coarsening approach to hypergraph partitioning. arXiv:2410.10875, 2024.
- D. Zhou, J. Huang, B. Schölkopf. Learning with hypergraphs: clustering, classification, and embedding. *NIPS*, 2006.